

Unidad 3



Programación de relés programables en FBD

Sistemas secuenciales programables





Índice

3.1. Lenguaje de programación FBD para LOGO

- 3.1.1. Lenguaje de programación FBD para LOGO
- 3.1.2. Conectores y constantes
- 3.1.3. Funciones generales (GF)
- 3.1.4. Funciones especiales (SF)

3.2. Programación básica

- 3.2.1. Circuitos realimentados
- 3.2.2. Circuitos con biestables

3.3. Funciones de tiempo (temporizadores)

- 3.3.1. Utilización de las señales de tiempo en las ecuaciones lógicas
- 3.3.2. Tipos de temporizadores

3.4. Función contador /descontador

3.5. Detección de flancos en señales digitales

- 3.5.1. Flanco positivo o ascendente
- 3.5.2. Flanco negativo o descendente
- 3.5.3. Uso de los flancos para evitar señales permanentes

3.6. Marcas internas

- 3.6.1. Uso de las marcas en secuencias básicas



Introducción

En esta unidad vamos a profundizar en la programación de los relés programables. Para ello, vamos a tomar como referencia el autómatas LOGO de la marca Siemens, que utiliza un lenguaje de programación llamado de forma general FBD.

En este lenguaje encontramos circuitos de bloques donde se utilizan principalmente entradas, salidas y marcas. Estos elementos basan su funcionamiento en las funciones lógicas que hemos estudiado en los temas anteriores.

Al finalizar esta unidad

- + Conoceremos las funciones de programación FBD para el LOGO
- + Identificaremos los bloques que usa este tipo de lenguaje de programación
- + Sabremos como se detectan los flancos de las señales digitales
- + Aprenderemos diferentes procesos industriales

3.1.

Lenguaje de programación FBD para LOGO

El **lenguaje FBD** se utiliza en la programación de relés usando las puertas lógicas que hemos estudiado en los temas anteriores. Gracias a este lenguaje se pueden resolver sistemas secuenciales.

En esta unidad vamos a indagar en la programación práctica **del relé programables LOGO** que pertenece a la marca Siemens, la cual utiliza el lenguaje FBD.



Imagen 1. Relé programable LOGO.
Fuente: <https://www.solucionesyservicios.biz>

3.1.1. Lenguaje de programación FBD para LOGO

Aunque a este lenguaje lo llamamos FBD, Siemens utiliza el término **FUP** para referirse a este lenguaje, por lo que a partir de ahora nosotros también lo adoptaremos.

Los bloques de funciones que se utilizan en el lenguaje FUP se pueden organizar en 3 categorías:

- > Conectores y constantes (CO).
- > Funciones generales (GF).
- > Funciones especiales (SF).

3.1.2. Conectores y constantes

Realizan la función de conectar el LOGO con su entorno, pudiendo guardar valores de señales y datos.

Podemos encontrar señales de entrada/salida, bits de las teclas del cursor, marcas, etc. Principalmente se usan las de entradas, salidas y marcas digitales.

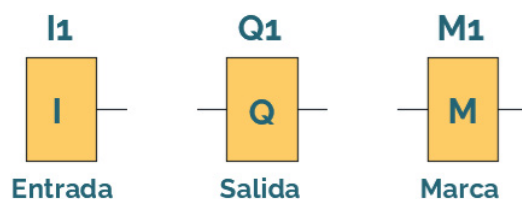


Imagen 2. Símbolos de conectores básicos

Como podemos observar, el bloque de salida solo tiene un terminal en la derecha, mientras que los bloques de marca y salida tienen dos terminales. Uno de estos dos terminales habilita la señal, mientras que el otro toma el valor lógico de dicha señal.

3.1.3. Funciones generales (GF)

Se trata de las funciones lógicas estudiadas que, combinándolas de la manera adecuada, pueden activar entradas, salidas y marcas de temporizadores y contadores.

Estas funciones generales tienen un número fijo de entradas y una sola salida.

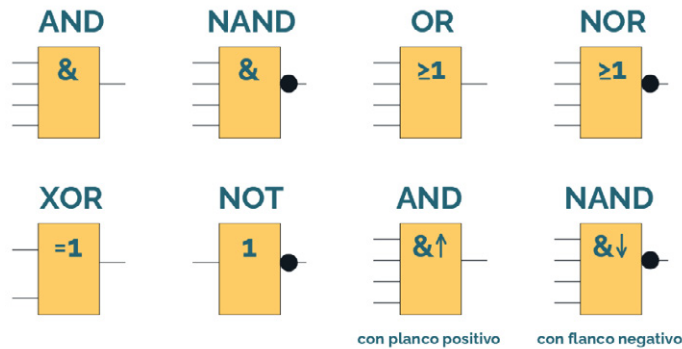


Imagen 3. Funciones generales

3.1.4. Funciones especiales (SF)

Estas funciones cumplen con un objetivo muy específico, como temporizar, contar, comparar, etc. Pueden coincidir tantas funciones especiales como el relé programable permita. Algunas de ellas son:

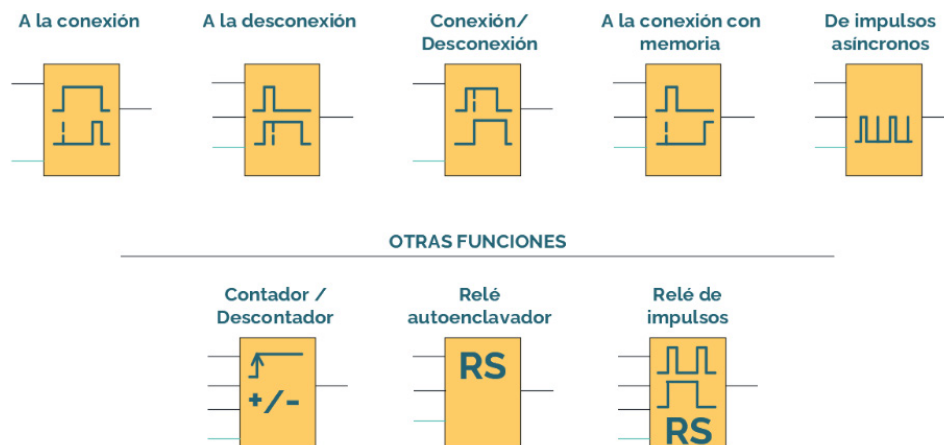


Imagen 4. Funciones especiales

3.2.

Programación básica

Para la programación en FUP vamos a ver algunos equipos que utilizan las reglas que hemos visto en las unidades anteriores ligadas con sistemas de realimentación y biestables.

3.2.1. Circuitos realimentados

En estos circuitos se seguirá el criterio por el que las señales de activación se corresponderán con la operación de suma a la **realimentación**, a diferencia de las de desactivación, que se niegan individualmente para luego multiplicarlas al bloque de activación. Esto permite crear condiciones de parada y puesta en marcha fácilmente. A continuación, podemos ver un ejemplo donde la salida se activa desde 3 puntos y se desactiva desde 2 de ellos:

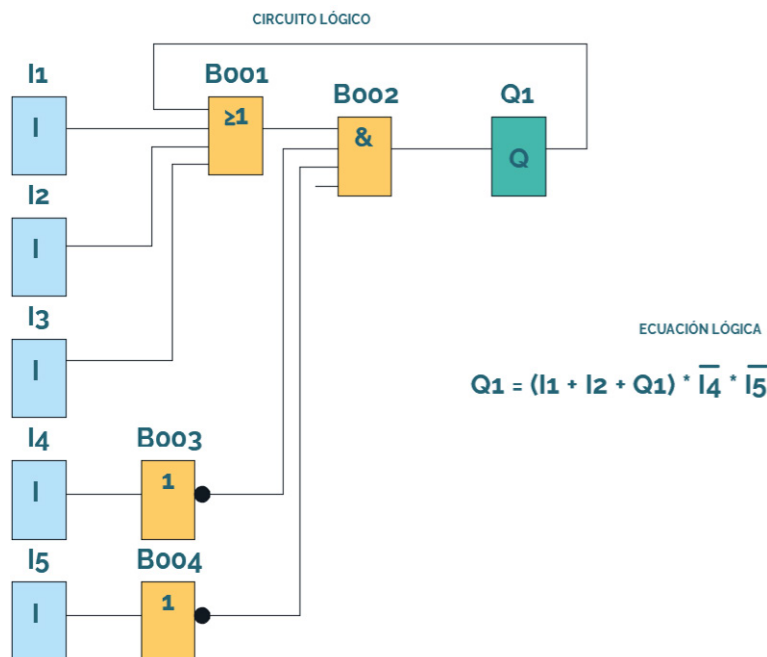


Imagen 5. Ejemplo de circuito con realimentación

Condicionar la activación de una salida a otra

Mediante este circuito se pretende que, para activar una segunda salida, la primera tenga que estar activada. De esta forma podemos encontrar dos casos.

Caso 1

En este primer caso vamos a observar como la primera salida solo afecta a la segunda salida en que, para activar la segunda, la primera tiene que estar activada.

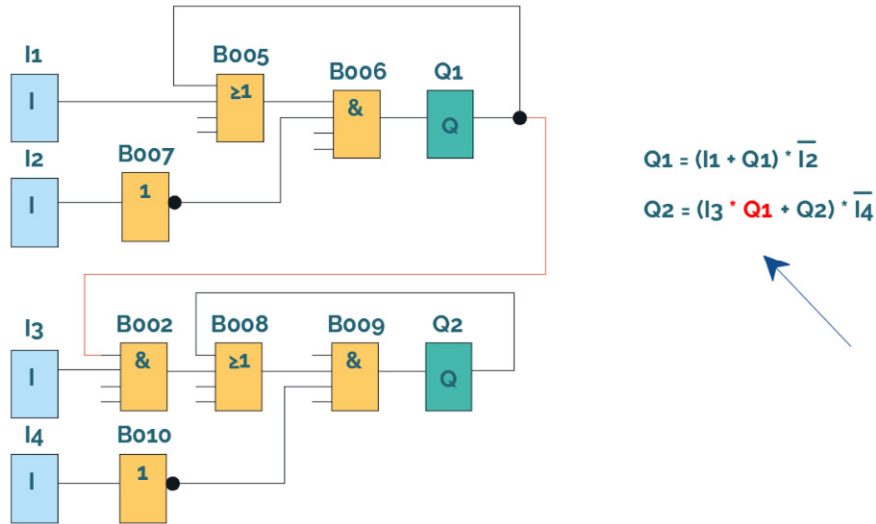


Imagen 6. Ejemplo de condicionante de activación

Como podemos ver, la salida 2 se activa a través de I3, pero teniendo que estar Q1 activa. Una vez activada Q2, la desactivación de Q1 no afecta a Q2.

Caso 2

En este caso, Q2 se verá afectada tanto por la activación de Q1, como por su desactivación.

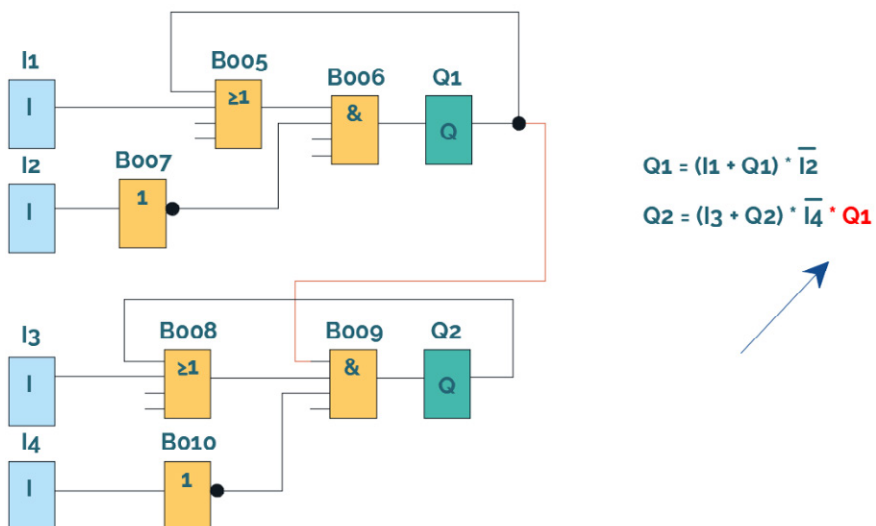


Imagen 7. Ejemplo de circuito condicionado para activación/desactivación

Como vemos ahora, si ambas salidas están activadas y Q1 se desactiva, esto afectará a la salida Q2, desactivándola también.

Condicionar la activación de una salida a la NO activación de otra

En este caso pasa lo mismo, pero para activar la segunda salida (Q2), la primera (Q1) tiene que estar desactivada.

Caso 1

Funcionalmente, la salida Q1 se niega y se multiplica a la ecuación de realimentación que afecta a la segunda salida.

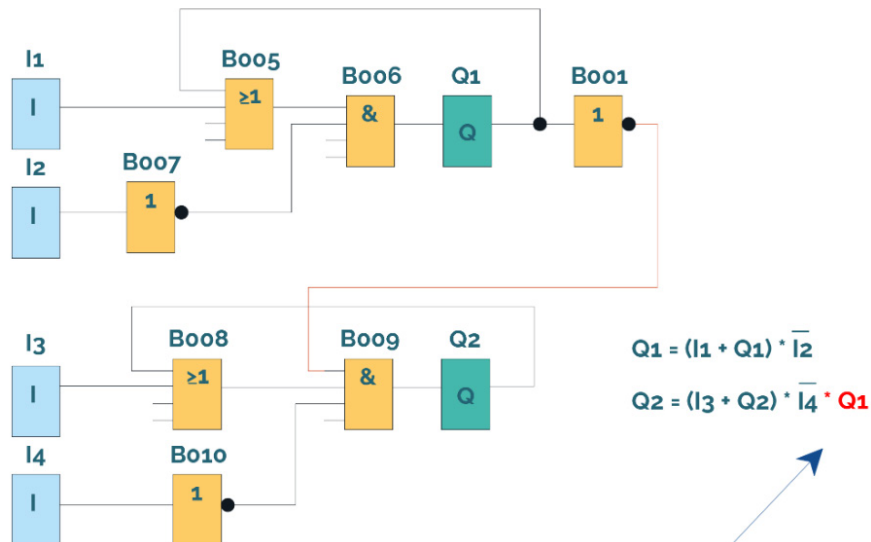


Imagen 8. Ejemplo de condición para la activación

En este circuito no puede coincidir que las dos salidas estén activadas a la vez. Para que Q2 esté activada, Q1 tiene que estar desactivada. En el momento en el que se active Q1, esta desactivará Q2.

Caso 2

En este caso vamos a ver como realizamos la misma etapa de negación de la primera salida (Q1), pero ahora se multiplica al bloque de la realimentación.

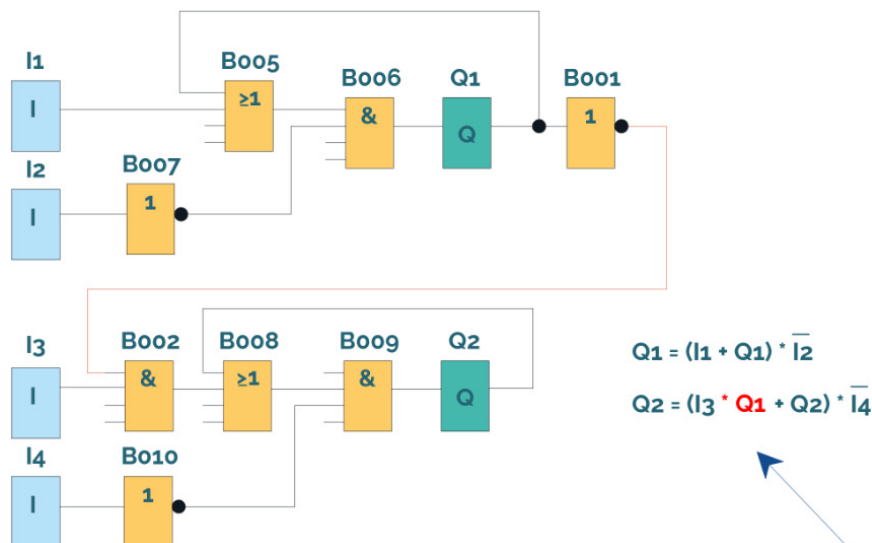


Imagen 9. Ejemplo de condición para activación/desactivación

De esta forma, se consigue que Q2 se siga viendo condicionada por Q1 en cuanto a la activación, no pudiendo activarse si Q1 lo está. Pero, sin embargo, Si hemos conseguido activar Q2 y, después, se activa Q1, esto no implicará la desactivación de Q2.

3.2.2. Circuitos con biestables

Los **biestables** simplifican la programación ya que, en vez de utilizar tres bloques como en los circuitos realimentados, solo se utiliza uno.

Si el biestable pertenece al autómatas LOGO, se denomina relé autoenclavador y está constituido por dos entradas: una para el SET y otra para el RESET.

Todo esto lo podemos ver en el siguiente ejemplo sencillo.

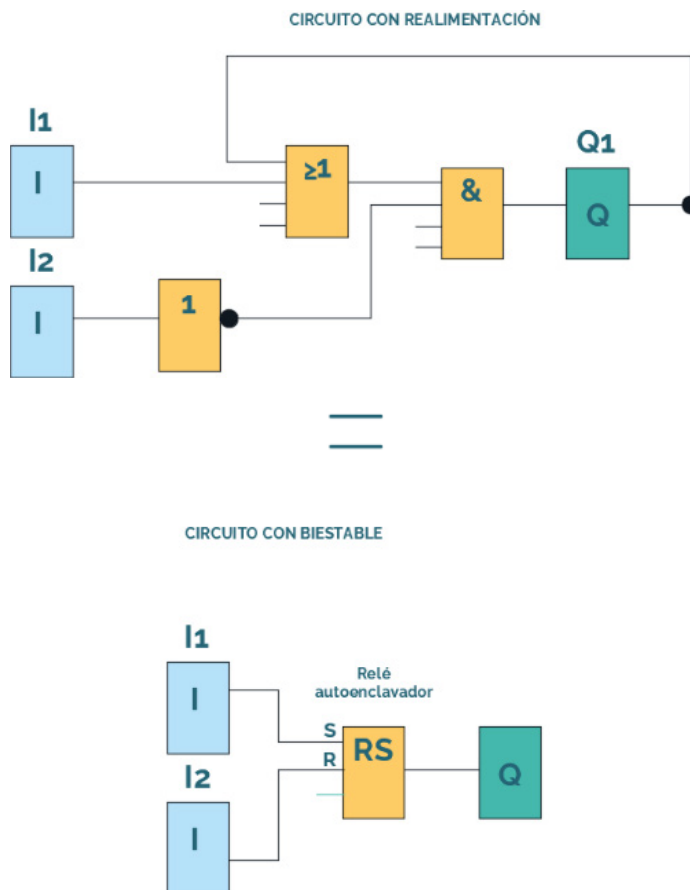


Imagen 10. Comparación de circuito con realimentación y con biestable

3.3.

Funciones de tiempo (temporizadores)

Los **temporizadores** son bloques de funciones especiales que nos posibilitan crear eventos en función de un tiempo prefijado.

Existen varios tipos diferentes de temporizadores, de los cuales cinco de ellos los vamos a ver en esta unidad. Entre ellos, prestamos especial atención a los de conexión y desconexión, puesto que guardan similitud con la lógica programada.

Estos bloques están compuestos por una o más entradas para realizar el control y una sola salida que conmuta dependiendo del tiempo preseleccionado y el modo de funcionamiento del temporizador.

Estos temporizadores dependen del tiempo, el cual se representa en el diagrama de bloques como una entrada de color verde.

3.3.1. Utilización de las señales de tiempo en las ecuaciones lógicas

A la entrada del temporizador (Trg) se le da una combinación lógica para su activación (T=).

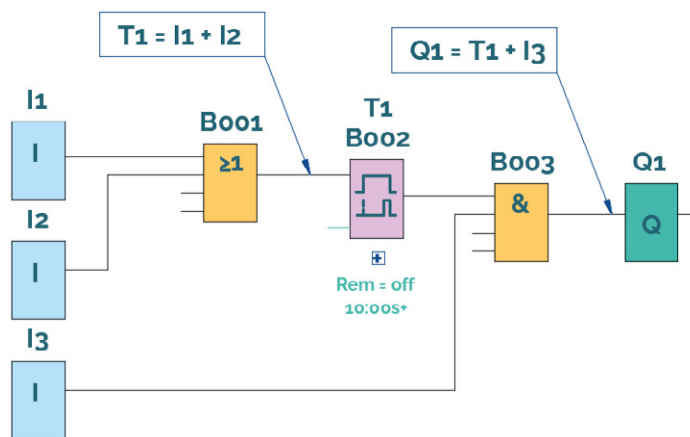


Imagen 11. Circuito lógico con temporizador

Como podemos ver en el circuito anterior, la salida se activará en el tiempo determinado por el temporizador, si la entrada I3 toma el valor lógico 1.

3.3.2. Tipos de temporizadores

A continuación, vamos a explicar los 5 tipos de temporizadores que se utilizan en el relé programable LOGO.

1. Temporizador con retardo a la conexión (TON)

Este temporizador activa su salida cuando se cumple el tiempo fijado. El temporizador empieza a contar cuando la entrada (Trg) toma el valor 1, pero si, durante ese tiempo, la entrada vuelve a tomar el valor 0, el relé se resetea. Si, por el contrario, la entrada mantiene el valor de 1 durante todo ese tiempo, entonces la salida se activa.

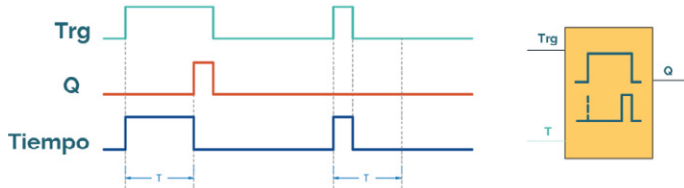


Imagen 12. Cronograma del temporizador a la conexión

2. Temporizador con retardo a la desconexión (TOF)

En este caso, cuando la entrada Trg toma el valor de 1, también lo hace la salida, pero cuando la entrada toma el valor de 0 pone en marcha el temporizador para que, tras pasar ese tiempo, la salida se desactive. El temporizador también tiene una entrada RESET para inicializar el temporizador.

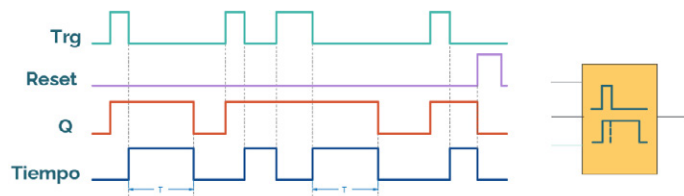


Imagen 13. Cronograma del temporizador a la desconexión

3. Temporizador con retardo a la desconexión con memoria

Se comporta de forma similar al retardo a la conexión. Cuando la entrada Trg toma el valor de 1 se dispara el temporizador, tras el cual la salida tomará el valor de 1. En este caso, la entrada se realiza por un pulso, por lo que, aunque la señal vuelva a tomar el valor de 0, no afecta a la salida. Sin embargo, el RESET es otra entrada que si puede devolver la salida al valor 0.

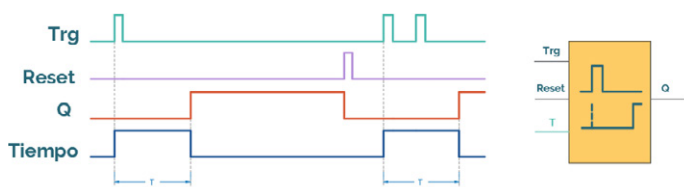


Imagen 14. Cronograma del temporizador a la desconexión con memoria

4. Generador de impulsos asíncronos

Activa su salida de forma intermitente con un tren de impulsos. En este temporizador se debe ajustar el tiempo que la salida va a estar activa (T_H), es decir, con valor 1 y el tiempo que la salida va a tomar valor 0 (T_L). Entonces cuando activemos la En (1), T_H y T_L se pondrán en alto o bajo según lo que hayamos programado hasta que se active el Inv, el cuál hará que estos valores se intercambien.

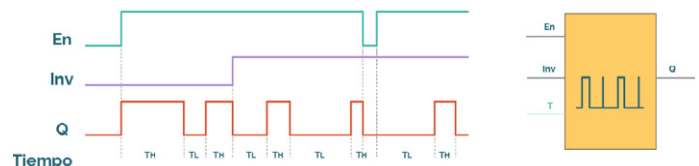


Imagen 15. Cronograma del generador de impulsos asíncrono

5. Temporizador con retardo a la conexión/desconexión

Con este temporizador se consigue el mismo resultado que si se conectan en serie un temporizador con retardo a la conexión y otro con retardo a la desconexión. La respuesta de la salida en este caso es que cuando la entrada toma el valor de 1, un temporizador se pone en marcha para activar la salida y, cuando la entrada toma el valor de 0, otro temporizador se pone en marcha para desactivar la salida.

3.4.

Función contador /descontador

La función "contador" posibilita registrar acontecimientos que se producen en sus entradas. La información se almacena en forma de número, por lo que cuando la entrada alcance dicho número, la salida se activa.

El contador del autómatas LOGO puede contar y descontar. Su control se realiza por medio de 3 entradas y 1 parámetro:

- > **RESET:** pone el contador a 0, desactivando la salida.
- > **CNT:** aumenta o disminuye el valor del contador cada vez que hay un pulso.
- > **DIR:** si tiene el valor 0, los pulsos en la entrada CNT aumentan el valor del contador. Si, por el contrario, el valor que tiene es 1, los pulsos disminuyen el valor.
- > **PAR:** es el valor prefijado como límite para que se active la salida del contador.

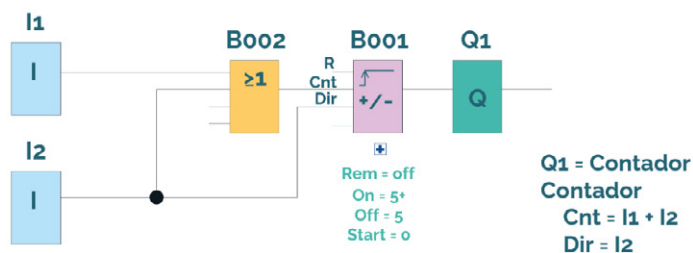


Imagen 16. Contador/descontador en LOGO

Un contador debe tener tantas ecuaciones lógicas como entradas del mismo se utilicen.

3.5.

Detección de flancos en señales digitales

Las funciones de flanco se utilizan para saber cuándo se produce un cambio en la variable digital. Este cambio en la variable ocasiona un impulso corto.

Los flancos pueden ser positivos o negativos.



Imagen 17. Flancos de una señal digital

Para el lenguaje FUP, los flancos ascendentes van ligados a la función lógica AND y los flancos descendentes a la función lógica NAND. En las dos situaciones, la salida está flanqueada con el signo correspondiente y se representa con una flecha.

3.5.1. Flanco positivo o ascendente

Se produce cuando la señal cambia del valor 0 al 1. Como hemos comentado antes, la señal se mantiene en 1 un periodo corto de tiempo.



Imagen 18. Señal de flanco positivo

3.5.2. Flanco negativo o descendente

Al contrario que el flanco positivo, este detecta cuando la señal cambia del valor 1 al 0.



Imagen 19. Señal de flanco negativo

En las ecuaciones lógicas también se representan los flancos mediante flechas verticales.

3.5.3. Uso de los flancos para evitar señales permanentes

La coexistencia de señales permanentes puede generar problemas de funcionamiento, por ejemplo, en sistemas secuenciales complejos, donde estas señales interfieren entre sí. La solución para este problema es implementar los flancos, de forma que se active la salida mediante un impulso, sin que el sensor que crea la acción se quede activado.

3.6.

Marcas internas

Las marcas o bits auxiliares son usadas en las operaciones internas de la memoria del autómata. En cuanto a su funcionamiento, no es muy diferente a la de los relés auxiliares de la lógica cableada.

Funciona igual que las salidas, pero en este caso no se realizan conexiones físicas para sacar la información fuera del PLC. En el lenguaje FUP las marcas se representan con la letra M y un número, por ejemplo, M1.

Las salidas pueden estar influenciadas por una o varias marcas como podemos ver en el siguiente ejemplo.

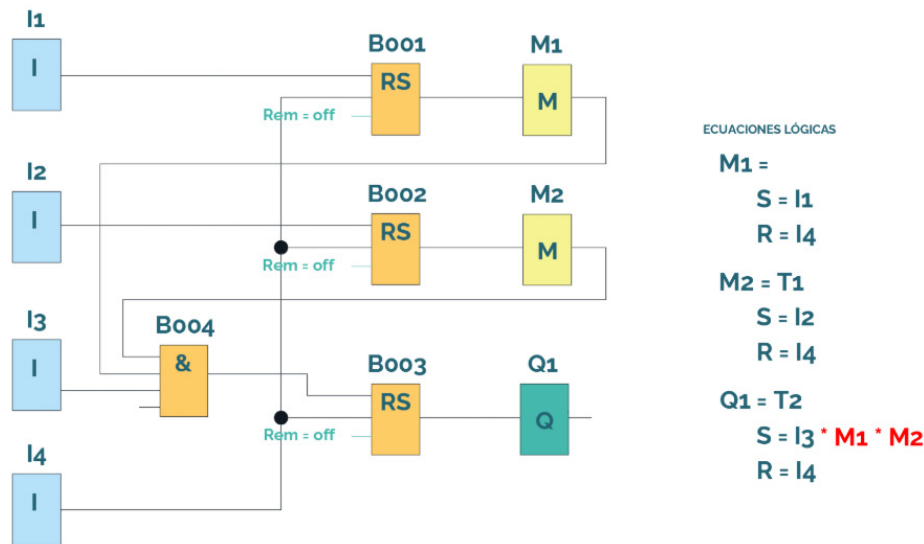


Imagen 20. Circuito donde la salida está influenciada por marcas

Como podemos ver, la salida Q1 está influenciada por las marcas M1 y M2. La salida y las marcas se desactivan simplemente por la entrada I4.

El número de marcas en un relé programable dependerá de la marca y del modelo. Estas marcas también pueden realizar funciones especiales. Por ejemplo, en el autómata LOGO, en el cual nos estamos centrando, encontramos una marca (M8) que genera un flanco positivo cuando el autómata se pone en marcha. Esta marca es necesaria para poder realizar más acciones después de que se inicie el sistema. En el siguiente esquema podemos ver como se representaría dicha marca.

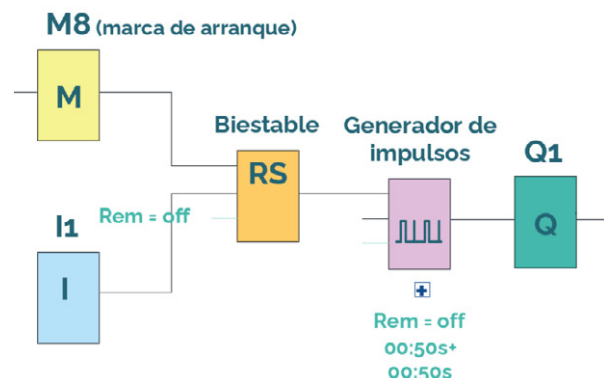


Imagen 21. Ejemplo de la marca M8 en el autómata LOGO

3.6.1. Uso de las marcas en secuencias básicas

En los autómatas programables el direccionamiento de las salidas es unívoco. Esto significa que únicamente se pueden llamar una vez en el programa. Sin embargo, es común conmutarlas en distintas situaciones de una misma secuencia. Para esto se plantea el programa en función de estados controlados por marcas.

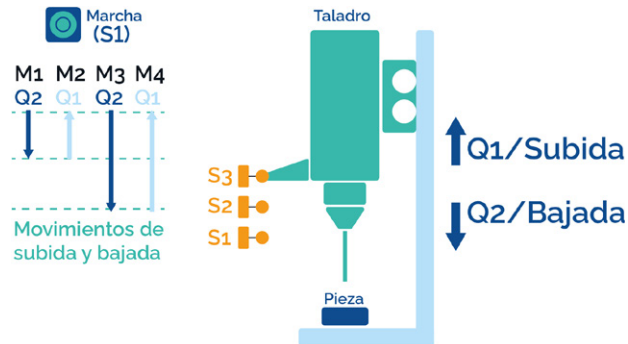


Imagen 22. Taladro de desahogo

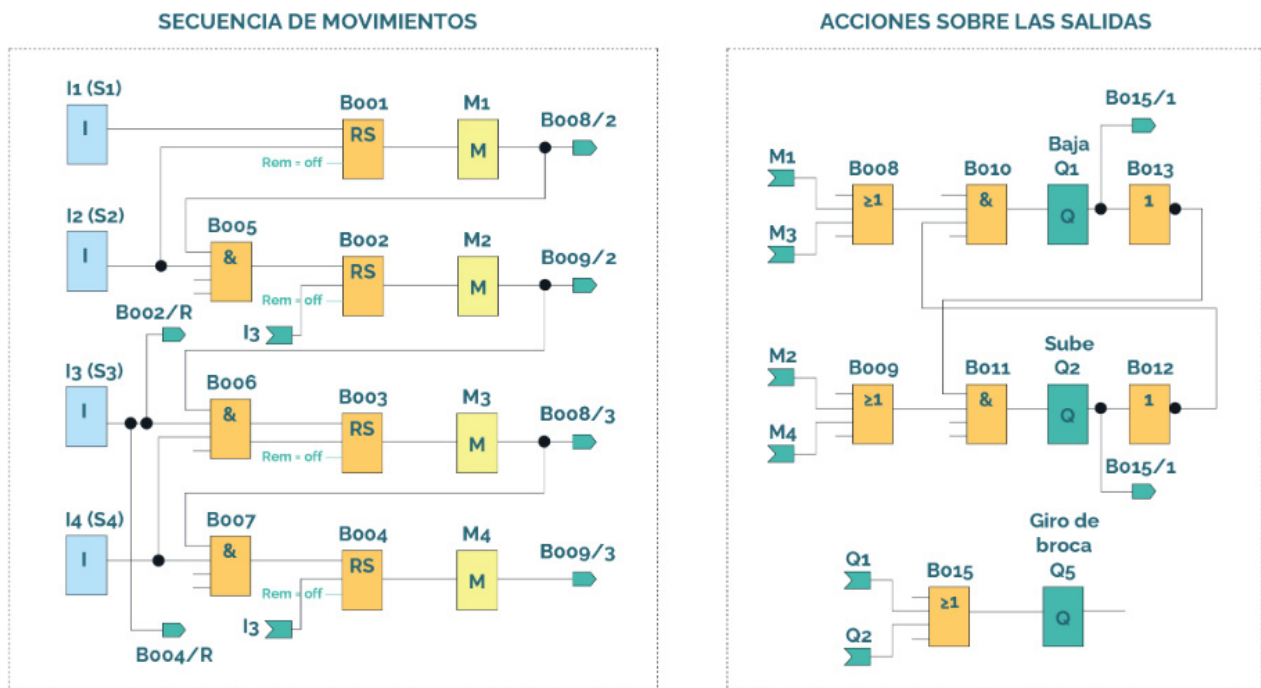


Imagen 23. Ejemplo de programa FUP con marcas



 www.universae.com

